FAKULTA STROJNÍHO INŽENÝRSTVÍ ústav automatizace a informatiky

ÚSTAV AUTOMATIZACE A INFORMATIKY

VAI		
Jméno: Filip Mironov		ID studenta: 239151
Datum zadání: 7.11.2024	Datum odevzdání: 5.2.2025	Studijní skupina: 3pAIŘ/1
Název úlohy: **Projekt – implementace algoritmu umělé inteligence**		

ZADÁNÍ

Vypracování programu využívající vámi vybranou metodu (metody) umělé inteligence pro řešení vybrané úlohy.

Programovací jazyk je na vaší volbě, odevzdán bude zdrojový i spustitelný kód.

Dále bude odevzdána technická zpráva obsahující úvodní list, teoretický popis řešené úlohy, metod pro její řešení a dále popis vytvořeného sw včetně závěru a použitých zdrojů. Technická zpráva by měla mít minimálně 4 strany a bude uložena ve formátu PDF.

ÚVOD

Algoritmus umělé inteligence bude implementován na hře „Maze escape“. V rámci předmětu VCP byl vytvořen závěrečný projekt obdobného charakteru. Tento projekt zahrnoval vytvoření statického bludiště pomocí objektů v programu C++. Celé bludiště (hrací plocha) musela být definována staticky, před spustěním programu. Výstup byl realizován pouze v konzoli bez grafického rozhraní. V rozboru bylo navrženo budoucí vylepšení v podobě automatické generace bludiště.

Aktuální varianta programu by měla splňovat následující parametry:

1) Grafická vizualizace hry.
2) Automatická generace bludiště.
3) Kontrola, jestli je daná varianta řešitelná pomocí aspoň jednoho algoritmu umělé inteligence.

4) Dynamická nápověda, která hráči nastíní nejlepší řešení bludiště v závislosti na jeho stávající poloze.

Pro realizaci bodu 2) byl zvolen informovaný algoritmus A* (A-star) a neinformovaný algoritmus Djikstra. Bod č. 4) bude realizován pomocí algoritmu A*.

A* je jeden z nejefektivnějších algoritmů pro hledání nejkratší cesty v grafech, například v herních mapách nebo navigačních systémech. Kombinuje Dijkstrův algoritmus (zaručeně najde nejkratší cestu) s informovaným vyhledáváním pomocí heuristiky, což ho činí rychlejším a efektivnějším [1].

A* využívá tento vzorec pro ohodnocení uzlů [1]:

- F = G + H
    - F – celková cena uzlu
    - G – vzdálenost od startovního uzlu
    - H – heuristika (odhadem zbývající vzdálenost k cíli)

Algoritmus vždy vybírá k rozšíření uzel s nejnižší hodnotou F, čímž se snaží minimalizovat celkovou délku cesty [1].

Dijkstrův algoritmus prochází a rozšiřuje všechny možnosti bez heuristiky, potažmo optimalizace, oproti A* nalezne nejkratší cestu do všech uzlů [1].

Program bude následně otestován na několika variantách a bude zohledněna jeho výpočová náročnost v závislosti na zvolené metodě kontroly realizovatelnosti [1].

1 PROSTŘEDÍ TVORBY PROGRAMU A IMPLEMENTACE ŘEŠENÍ

Pro realizaci kódu bylo použito programovacího jazyka Python, verze 3.12.0., v době tvorby programu nejaktuálnější verzí. Jako prostředí sloužila platforma Pycharm, společnosti JetBrains, za použití studentské licence. Důvodem této volby je především zkušenost s tímto prostředím a podpora českého programátorského průmyslu. Použitá verze Pycharm: 2024.3.1. Žádné další programovací jazyky, jak už z hlediska symbiózy nebo alternativní implementace, nebyly využity.

Program vyžaduje knihovny „pygame“ a „random“. Knihovna pygame, verze 2.6.0, zaručuje grafickou implementaci hry. Knihovna random je součástí standartní knihovny programu python. Při tvorbě bylo zanecháno defaultní nastavení Pycharm, využívající pip (Pip Installs Packages) verze 23.2.1. Jediným alternativním nástrojem, který přispěl ke tvorbě byl add-on black, verze 24.8.0.

Python je objektově orientovaný jazyk, z tohoto důvodu bylo využito objektů a řady metod. V programu je využito techniky „backtracking“. Pro ulehčení implementace bylo využito datové struktury „list“, která je defaultním nástrojem Pythonu.

2 POPIS SW

2.1 Objekty

Node

Každé pole (čtvercové políčko bludiště) je instance třídy Node, obsahuje informaci o jeho poloze v prostoru (x, y), typu ('wall', 'path', 'player', 'finish' a 'help'), pointery na okolní 4 pole a údaje pro algoritmy umělé inteligence (náklady na cestu, heuristiku a pointer na nejlevnější mateřské pole).

```
class Node:
    def __init__(self):
        self.pos_x, self.pos_y = None, None
        self.type = 'wall'
        self.up, self.down, self.left, self.right = None, None, None, None
        self.bestPrev = None
        self.cost = None
        self.heuristic = None
```

Maze

Objekt „Maze“ tvoří hrací pole z instancí třídy Node. Zahrnuje globální proměnné „running“, pro cyklické spuštění obnovování okna v pygame. Údaje o velikosti okna a pole. Proměnné typu list „NodePointers“ a „NodeTypes“ s odpovídajícími indexi zjidnodušují prohledávání grafu v určitých případech. V budoucnu je možno nahradit. Pointery StartNode a EndNode odkazují na instance Nodu s typem „player“ a „finish“.

```
class Maze:
    def __init__(self, screen_width, screen_height):
        self.running = True

        self.scrn_w, self.scrn_h = screen_width, screen_height
        self.Nodesize = None

        self.NodePointers = []
        self.NodeTypes = []

        self.StartNode = None
        self.EndNode = None
```

2.2 Metody

build_maze

Tato metoda slouží k automatické generaci bludiště v závislosti na vstupní proměnné size. Celkově má pak bludiště rozměr size*size. Je zde zajištěno, že všechny okraje jsou typu „wall“, aby hráč nemohl vystoupit z bludiště. Pomocí random je náhodně vygenerován obsah z možností „wall“ a „path“, každá s pravděpodobností 0,5.

Zde je také realizováno vytvoření hran grafu a vyplnění dalších dynamických promměných jako poloha x a y. Instance jsou dokumentovány v listech „NodePointers“ a „NodeTypes“.

Posledním krokem je navázání pointerů „EndNode“ a „StartNode“.

Euklid

Slouží k výpočtu heuristiky na základě koordinátů x a y mezi vstupním polem a polem „EndNode“.

Astar

Zde je implementován algoritmus umělé inteligence A*.

V prvním kroku jsou anulovány proměnné instancí, které mohou mít zanechanou stopu z předchozího užití metody. Jde zde především o cenu cesty „cost“ a pointer na nejlevnějšího předchůdce „bestPrev“.

Dále jsou definovány proměnné nutné k realizaci A*, jako closed a open list. Náklad na cestu do startovního nodu je nastaven na 0. Startovní nod je zařazen do listu open.

V třetím kroku jsou cyklicky procházeny nody z listu open, prioritně ty s nejnižší hodnotou podle předpisu F=G+H. Pokud sousedé nejsou typu „wall“ a nejsou v listu closed, následuje kontrola přítomnosti v listu open. Pokud se zde nenachází, jsou přidáni a doplněny informace o nákladu a nejlevnějším předchůdci. Pokud již jsou v listu open, na základě ceny jsou jejich údaje přepsány (levnější cesta), nebo zanechány (zhoršení).

Pokud se v listu closed nachází instance „EndNode“, tak se pomocí backtrackingu generuje nejlevnější cesta. V případě, že cesta neexistuje, list open je prázdný, je výstupem False.

Djikstra

Funguje na podobném principu jako metoda Astar, s rozdílem absence heuristiky. Procházeny jsou všechny nody, bez priority a výstup je podmíněn tím, že list open je prázdný.

move

V závislosti na vstupu z klávesnice upravuje pointer „StartNode“

run

Zajišťuje grafickou realizaci hry a interakci s hráčem.

Všechny metody jsou detailněji okomentované v kódu.

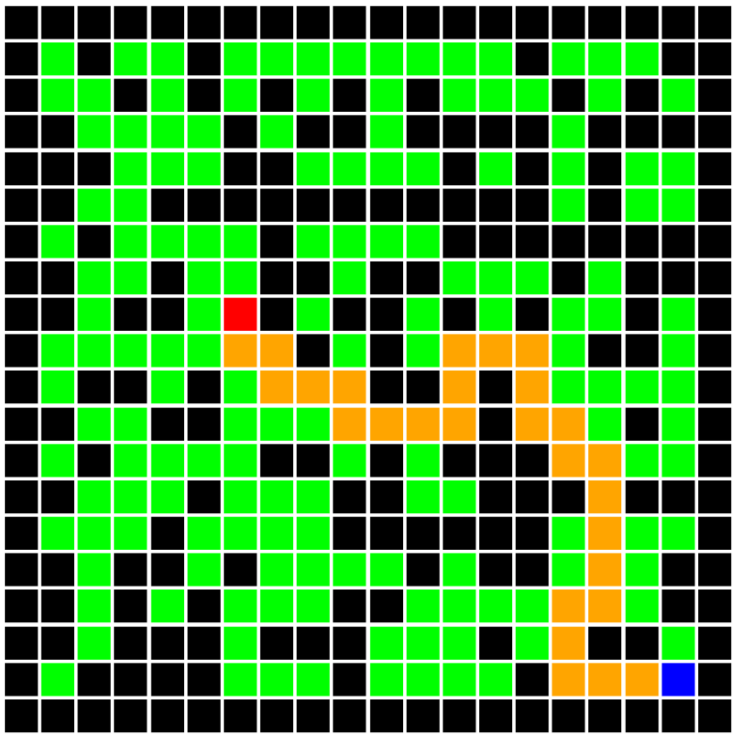
3 BĚH PROGRAMU

V těle programu probíhají následující kroky:

1) Spustí se časovač.
2) Je generováno bludiště
3) Časovaná kontrola pomocí A*.
4) Časovaná kontrola pomocí Djikstry.
5) Pokud není bludište řešitelné, vrací se ke kroku 2).
6) Pokud ano, je vypsáno v konzoli počet generovaných variant a celkový čas, využit ke kontrole jednotlivými metodami

```
Elapsed time to find solvable Maze: 19.17 seconds
Number of possible Mazes generated: 2202
Elapsed time to find solvable using Djikstra: 0.12 seconds
Elapsed time to find solvable using A*: 0.14 seconds
```

7) Spuštění hry.
    a. Vstupy „w“, „s“, „a“, „d“ se hráč pohybuje po mapě.
    b. Podržením „mouse1“ se hráči dynamicky zobrazí nápověda v závislosti na jeho aktuální poloze (oranžově).
    c. Červené pole = pozice hráče.
    d. Modré pole = cíl.
    e. Zelené pole = průchozí. Černé = neprůchozí



4 BUDOUCÍ DEVELOPMENT A MOŽNÉ IMPLEMENTACE

Program je v této verzi limitovaný především složitou generací bludiště. Optimalizace tohoto procesu povede k značné úspoře výpočetní náročnosti.

Tento projekt má sloužit jako demonstrace implementace a otestování určitých algoritmů. Z testování lze odvodit, že základ programu (generace, vizualizace) mají podstatně vyšší vliv na výpočetní náročnost generování grafu a jejich optimalizací lze dosáhnout vyšší úspory, než použití složitějšího algoritmu umělé inteligence.

ZÁVĚR

Cílem této práce bylo seznámení se ze základy algoritmů umělé inteligence, implementace zvolené metody a následná dokumentace projektu.

Byla vytvořena hra „Maze escape“ v programu Python. Cílem hry je dostat hráče na cílovou pozici z výchozí pozice. Součástí programu je náhodná generace herní plochy a následné ověření toho, zda je vygenerovaná varianta řešitelná. Ověření probíhá pomocí algoritmů A* a Djikstra, jejichž výpočetní náročnost je sledována. Hra je obohacena o možnost zobrazení nápovědy, která hráči zvýrazní nejkratší cestu do cíle z jeho aktuální polohy.

SEZNAM POUŽITÝCH ZDROJŮ

[1] SWIFT, Nicholas. *Easy A* (star) Pathfinding* [online]. MEDIUM, únor 2017 [cit. 2025-02-05]. Dostupné z: https://medium.com/@nicholas.w.swift/easy-a-star-pathfinding-7e6689c7f7b2.